

Ravindu - Bilingual Security Video Guide

Sinhala action guide + English speech + code segments + test evidence

Member	P.A.R.B. Subasingha (Ravindu)
Original website	http://localhost:5174/login
Secure website	http://localhost:5173/login
Assigned scope	V-01, V-02, V-03, V-05, V-06 and Google OIDC

How to use this PDF

Ravinduගේ Security Video Script - සිංහලෙන්

මෙය video එක record කරන වලොවේ screen එක බලමින් කියවන්න පුළුවන් සරල script එකකි. හැම vulnerability එකකටම මුලින් Original website එක පෙන්වන්න. ඊට පස්සේ Secure website එක පෙන්වන්න. Original එකේ තිබුණු අවදානම සහ Secure එකේ කළ fix එක පැහැදිලිව compare කරන්න.

English introduction for the video

Ravindu - English Video Speech Script

Use this file for the spoken English voice-over. Use Ravindu-video-script-Sinhala.md separately as the Sinhala step-by-step guide for clicking, typing and taking screenshots.

V-01 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

V-01 - Public Registration / Administrator Token Creation

1. Original website එක පෙන්වීම

1. Original tab එක open කරලා `http://localhost:5174/login` යන්න.
2. Login form එක භිස්ව තිබෙන විට cursor එක Username field එක උඩට ගෙන screenshot එක ගන්න:

V01-01-original-login-empty.png

3. Username එක cashier.colombo ලෙස type කරලා screenshot එක ගන්න:

V01-02-original-username-filled.png

4. Local demo password එක type කරලා field එක masked බව පෙන්වමින් screenshot එක ගන්න:

V01-03-original-password-filled.png

5. Sign In click කරලා dashboard එක load වුණාට පස්සේ employee-access badge එක පෙන්වමින් screenshot එක ගන්න:

V01-04-original-dashboard.png

Original ගැන කියන වචන

"Original backend එකේ /api/register route එක public route එකක් ලෙස තිබුණා. Authentication එකක් නැති කෙනෙකුට registration request එකක් යවා role එක admin ලෙස ඉල්ලන්නත් token එකක් ලබාගන්නත් හැකියාව තිබුණා. ඒකෙන් attacker කෙනෙකුට administrator access ලබාගැනීමට උත්සාහ කළ හැකි නිසා මේක broken access control vulnerability එකක්. මේ අවදානම පෙන්වන්න exploit request එක submit කරන්නේ නැහැ; source code එක පමණක් පෙන්වනවා."

2. Secure website එක පෙන්වීම

1. Secure tab එකට Ctrl + Tab කරලා http://localhost:5173/login open කරන්න.

2. Login form එකේ Google button එකත් තිබෙන secure login screen එක screenshot කරන්න:

V01-05-secure-login.png

3. Secure API route file එක open කරලා registration route එක auth:sanctum, active සහ admin middleware ඇතුළත තිබෙන කොටස පෙන්වන්න. Screenshot:

V01-06-secure-admin-route-group.png

4. tests/Feature/Security/AccessControlTest.php එකේ test_registration_is_not_reachable_without_authentication test එක පෙන්වන්න. assertStatus(401) සහ database එකේ attacker user කෙනෙක් නැති බව පෙන්වමින් screenshot කරන්න:

V01-07-secure-registration-test.png

Secure ගැන කියන වචන

"Secure version එකේ /register public section එකෙන් ඉවත් කරලා administrator-only route group එක ඇතුළට දමා තිබෙනවා. ඒ නිසා unauthenticated request එකකට 401 response එකක් ලැබෙනවා. Authenticated employee කෙනෙකුටත් registration කරන්න බැහැ. Regression test එක 401 status එකත් attacker account එක database එකට නොඑන බවත් verify කරනවා."

පෙන්විය යුතු code point

```
Route::middleware(['auth:sanctum', 'active'])->group(function () {
    Route::middleware('admin')->group(function () {
        Route::post('/register', [AuthController::class, 'register']);
    });
});
```

V-01 - Public Registration and Administrator Token Creation

Original website

[Open `http://localhost:5174/login`. Show the empty login form, then enter the demo username and password. Click Sign In and show the employee dashboard.]

"I am now using the Original website. I am signing in with the employee account `cashier.colombo`. The password is kept masked for security."

[Show the Original dashboard and employee access information.]

"The important problem is in the backend registration route, not in this login screen. In the Original backend, the registration endpoint was publicly reachable. An unauthenticated person could send a registration request, request the administrator role, and receive an authentication token."

This creates a broken access-control vulnerability. An attacker could try to create an administrator account without already being an authorized administrator. I am not submitting that exploit during the live demonstration. I will show the vulnerable route and the security test instead."

Secure website

[Press `Ctrl + Tab` and open `http://localhost:5173/login`. Show the Secure login page.]

"I am now switching to the Secure website. The same employee login is used here, and the password remains masked."

[Show the secured route group in the API source code.]

"In the Secure version, the registration endpoint is no longer public. It is protected by Sanctum authentication, the active-user check, and administrator authorization."

```
Route::middleware(['auth:sanctum', 'active'])->group(function () {
    Route::middleware('admin')->group(function () {
        Route::post('/register', [AuthController::class, 'register']);
    });
});
```

[Show `AccessControlTest.php` and the 401 assertion.]

"The regression test sends an unauthenticated request and expects HTTP 401. It also verifies that an attacker user is not created in the database. Therefore, the vulnerability is fixed at the backend authorization layer, rather than being hidden only in the frontend."

Key code segments and file locations

Key secure route protection

File: `demo/secure/pubudu-pos-api-secure/routes/api.php` Lines: 80 and 187-193

```
Route::middleware(['auth:sanctum', 'active'])->group(function () {
    Route::middleware('admin')->group(function () {
        Route::post('/register', [AuthController::class, 'register']);
    });
});
```

Security test result

File: `demo/secure/pubudu-pos-api-secure/tests/Feature/Security/AccessControlTest.php` Lines: 20-35

```
$response = $this->postJson('/api/register', $payload);
$response->assertStatus(401);
$this->assertDatabaseMissing('users', ['username' => 'attacker']);
```

V-02 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

V-02 - Missing Role Authorization / Profit Report

1. Original website එකේ bug එක

1. Original tab එකට මාරු වලො login කරලා තිබෙන cashier account එකනේ:

`http://localhost:5174/sales/profit` open කරන්න.

2. Report එක load වන තනේ ඉන්න. Revenue, Cost, Profit, Margin values පතේවන තැන screenshot කරන්න:

`V02-01-original-employee-profit-report.png`

3. ඒ values පතේවන තැන් cursor එකනේ point කරලා මේ වචන කියන්න.

Original ගැන කියන වචන

“මම මතෙන login වලො ඉන්නේ administrator කෙනෙකේ ලෙස නොවෙයි, Colombo cashier කෙනෙකේ ලෙසයි. ඒත් original website එකේ `/sales/profit` URL එක direct open කළාම Revenue, Cost, Profit සහ Margin report එක පතේවනවා. Login වී සිටීම authentication එකක් පමණයි; administrator permission එකක් තිබෙනවා කියන එක නොවෙයි. Original frontend route එකේ ProtectedRoute පමණක් තිබුණු නිසා role check එක අමතක වී තිබුණා.”

2. Secure website එකේ fix එක

1. Secure tab එකට මාරු වලො `http://localhost:5173/sales/profit` open කරන්න.

2. Employee account එකට report එක නොපතේවා dashboard එකට redirect වෙන බව පතේවන්න. Dashboard URL සහ employee-access badge එක point කරලා screenshot කරන්න:

`V02-02-secure-profit-denied.png`

3. Secure frontend `src/App.jsx` එකේ ProtectedRoute `adminOnly={true}` පතේවන්න:

`V02-04-secure-admin-route.png`

4. Secure API route file එකේ financial report routes admin middleware group එක ඇතුළතේ තිබෙන බවත්, `AccessControlTest.php` එක employee request එකකට 403 expect කරන බවත් පතේවන්න:

`V02-05-secure-access-control-test.png`

Secure ගැන කියන වචන

“Secure version එකේ frontend route එකට `adminOnly={true}` දමා employee කෙනෙකුට report page එක open වීම නවත්වලා තිබෙනවා. ඒක පමණක් නොවෙයි, backend API එකේ financial report routes administrator middleware group එක ඇතුළතේ දමා තිබෙනවා. ඒ නිසා address bar එකනේ direct URL එකක් දුන්නත් UI එක bypass කරලා data ගන්න බැහැ. Test එක employee request එකකට 403 response එකක් සහ administrator request එකකට 200 response එකක් verify කරනවා.”

පැමිණිලිය යුතු code point

```
<Route
  path="/sales/profit"
  element={
    <ProtectedRoute adminOnly={true}>
      <Layout>
        <ProfitReport />
      </Layout>
    </ProtectedRoute>
  }
/>
```

English speech - read this in the video

V-02 - Missing Role Authorization and Profit Report Exposure

Original website

[Switch to the Original tab and open `http://localhost:5174/sales/profit`. Show the Revenue, Cost, Profit and Margin values.]

"I am still logged in as a Colombo cashier, not as an administrator. However, in the Original website I can open the profit-report URL directly."

[Point the cursor to the financial values.]

"The page exposes revenue, cost, profit and margin information to an ordinary employee. Being authenticated only proves the identity of the user. It does not prove that the user has administrator permission."

The Original frontend protected the route from unauthenticated users, but it did not perform an administrator role check. This is a missing role-authorization vulnerability. Sensitive financial data must be protected by the backend as well as by the frontend."

Secure website

[Press `Ctrl + Tab` and open `http://localhost:5173/sales/profit`. Show the redirect or denied employee view.]

"Now I am trying the same URL in the Secure website with the same employee account. The employee is redirected away from the profit report, so the sensitive report is not displayed."

[Show the Secure frontend route.]

```
<Route
  path="/sales/profit"
  element={
    <ProtectedRoute adminOnly={true}>
      <Layout>
        <ProfitReport />
      </Layout>
    </ProtectedRoute>
  }
/>
```

"The frontend route now requires administrator access with `adminOnly`."

[Show the API route group and the access-control test.]

"The backend financial routes are also inside an administrator middleware group. This is important because a user must not be able to bypass the frontend by calling the API directly. The automated test expects HTTP 403 for an employee and HTTP 200 for an authorized administrator."

Key code segments and file locations

Frontend administrator guard

File: demo/secure/pubudu-pos-front-end-secure/src/App.jsx Lines: 176-185

```
<Route
  path="/sales/profit"
  element={
    <ProtectedRoute adminOnly={true}>
      <Layout><ProfitReport /></Layout>
    </ProtectedRoute>
  }
/>
```

Backend authorization result

File: demo/secure/pubudu-pos-api-secure/routes/api.php Lines: 80, 187 and 264-274

```
Route::middleware(['auth:sanctum', 'active', 'admin'])->group(function () {
    // financial report endpoints
});

// Employee request: 403 Forbidden
```

V-03 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාට ස කරන විදිහ

V-03 - Cross-Branch Customer IDOR / BOLA

1. Original website එකමේ bug එක

1. Original tab එකමේ <http://localhost:5174/customers> open කරන්න.
2. Employee badge එකමේ Kamal Silva (Colombo Cashier) සහ Colombo Main Branch පෙන්වන්න. Customer list එක load වූ පසු screenshot කරන්න:

V03-01-original-customer-list.png

3. Search field එක හිස්ව තිබෙන විට cursor එක field එක උඩට ගෙන screenshot කරන්න:

V03-02-original-search-empty.png

4. Search field එකට Dilani Wickramasinghe type කරන්න. Result එකමේ customer name සහ Kandy Branch පෙන්වෙන විට screenshot කරන්න:

V03-03-original-cross-branch-result.png

5. Edit හමේ Delete click කරන්න එයා. Result name එක සහ Kandy Branch එක cursor එකමේ පෙන්වන්න.

Original ගැන කියන වචන

“මමේ login වී සිටින user Colombo branch cashier කෙනෙක්. නමුත් search එකමේ Kandy Branch customer කෙනෙකුගේ record එක සහ credit information පෙන්වනවා. මමේ cross-branch object-level authorization failure එකක්, IDOR හමේ BOLA ලෙස හඳුන්වන්න පුළුවන්. User කතාට customer ID එකක් හමේ නමක් දැනගන්නමත් වෙනත් branch එකක record එකක් බලන්න හැකි වීම තමයි අවදානම.”

2. Secure website එකමේ fix එක

1. Secure tab එකමේ <http://localhost:5173/customers> open කරන්න.
2. Customer list එක load වූ පසු screenshot කරන්න:

V03-04-secure-customer-list.png

3. එම search field එකටම Dilani Wickramasinghe type කරන්න.

4. No customers found සහ එකම search text එක පනේවන වීඩියෝ screenshot කරන්න:

V03-05-secure-no-result.png

5. Code evidence ලෙස CustomerController.php හි visibleTo(\$request->user()), Customer.php හි ScopedToDepartment සහ concern එකේ department query එක පනේවන්න. Test එකේ cross-branch request එක 404 වන බව screenshot කරන්න:

V03-06-secure-controller-scope.png V03-07-secure-customer-model.png

V03-08-secure-department-scope.png V03-09-secure-cross-branch-test.png

Secure ගැන කියන වචන

"Secure version එකේ customer list එක සහ individual customer lookup එක දකිම employee department එක අනුව filter කරනවා. Colombo cashier කෙනෙක් Kandy customer කෙනෙක් search කළාම result එකක් නොලැබෙනවා. Backend එක visibleTo() scope එක භාවිතා කරනවා. Employee කෙනෙකුට වනේත් branch record එකක් ID එකෙන් direct ඉල්ලුවත් record එක තිබෙන බව හමු නොකර 404 response එකක් ලැබෙනවා. Test එක name සහ credit balance response එකේ නැති බවත් verify කරනවා."

පනේවිය යුතු code point

```
$customer = Customer::with('department')
->visibleTo($request->user())
->find($id);
```

English speech - read this in the video

V-03 - Cross-Branch Customer IDOR or BOLA

Original website

[Switch to the Original tab and open <http://localhost:5174/customers>. Show the Colombo cashier badge and customer list.]

"Next, I am demonstrating cross-branch customer-data access. The current user is a Colombo branch cashier."

[Search for Dilani Wickramasinghe. Show the Kandy Branch result.]

"When I search for this customer in the Original website, a Kandy Branch customer record is displayed to the Colombo cashier. The result can expose customer information and credit-related data from another branch."

This is an object-level authorization failure, commonly called IDOR or BOLA. The user is authenticated, but the application does not correctly check whether this particular customer object belongs to the user's permitted branch."

[Do not click Edit or Delete.]

"I will not edit or delete the record. I am only demonstrating that the cross-branch record is visible."

Secure website

[Press Ctrl + Tab and open <http://localhost:5173/customers>. Search for the same customer.]

"Now I am repeating the same search in the Secure website. The Colombo employee can still use the customer page, but the Kandy customer is not returned."

[Show the empty result or No customers found message.]

"This confirms that the search result is filtered by the employee's permitted department or branch."

[Show CustomerController.php, Customer.php, the department-scope concern, and the cross-branch test.]

"The backend applies the visibleTo scope before returning a customer. The model and department scope ensure that the query is restricted to the employee's branch."

```
$customer = Customer::with('department')
->visibleTo($request->user())
->find($id);
```

"Even if an employee guesses another customer ID and requests it directly, the backend does not reveal the record. The test expects a not-found response and verifies that the other branch's customer name and credit balance are not present in the response."

Key code segments and file locations

Department-scoped customer lookup

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/CustomerController.php Lines: 18-25 and 131-145

```
$customer = Customer::with('department')
->visibleTo($request->user())
->find($id);

if (! $customer) {
    return response()->json(['message' => 'Not found'], 404);
}
```

Reusable branch scope

File: demo/secure/pubudu-pos-api-secure/app/Models/Concerns/ScopedToDepartment.php Lines: 58-80

```
public function scopeVisibleTo(Builder $query, ?User $user): Builder
{
    return $query->where('department_id', $user?->department_id);
}
```

V-05 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කොටස කරන විදිහ

V-05 - Authentication Weaknesses

මමේ password attempts live demo එකමේ ඉතා සීමිතව පමණක් කරන්න. Password එක screen එකමේ පෙන්වන්න එපා. Toast message එක ඉක්මනින් disappear වුණොත් ඒක screenshot එකමේ පෙනෙනවා කියලා කියන්න එපා; code සහ tests වලින් exact result එක පෙන්වන්න.

1. Original website එක

1. Original tab එකමේ logout කරලා login page එක පෙන්වන්න. Screenshot:

V05-01-original-login-empty.png

2. Username ලෙස unknown.demo.user සහ harmless wrong password එකක් type කරලා screenshot කරන්න:

V05-02-original-unknown-input.png

3. Sign In click කරලා response state එක screenshot කරන්න:

V05-03-original-unknown-response.png

4. නැවත username ලෙස cashier.colombo දැලා එම wrong password එක type කරන්න. Screenshot:

V05-04-original-known-input.png

5. Sign In click කරලා response state එක screenshot කරන්න:

V05-05-original-known-response.png

6. Original AuthController.php code එක පෙන්වන්න. Unknown username එකට 404 message එකක්, known username එකට wrong password 401 message එකක් යන කොටස් පෙන්වන්න:

V05-09-original-enumeration-code.png

Original ගැන කියන වචන

"Original backend එක unknown username එකකට 'username does not exist' කියලා වනෙම 404 response එකක් දෙනවා. Existing username එකකට wrong password දුන්නාම 'password is incorrect' කියලා වනෙම 401 response එකක් දෙනවා. ඒ නිසා attacker කෙනෙකුට password guess කිරීමට කලින් valid usernames හඳුනාගන්න පුළුවන්. මේක username enumeration vulnerability එකක්. Brute-force attempts සඳහා login-specific rate limit එකක් තිබුණේ නැහැ."

2. Secure website එක

1. Secure tab එකේ secure login page එක පෙන්වන්න:

V05-06-secure-login-empty.png

2. unknown.demo.user සහ එම wrong password එක type කර submit කරන්න. Screenshot:

V05-07-secure-unknown-response.png

3. cashier.colombo සහ එම wrong password එක type කර submit කරන්න. Screenshot:

V05-08-secure-known-response.png

4. Secure AuthController.php හි dummy hash check සහ generic message එක පෙන්වන්න:

V05-10-secure-generic-login-code.png

5. routes/api.php හි throttle: login route එක සහ authentication tests පෙන්වන්න:

V05-11-login-throttle-route.png V05-12-enumeration-test.png V05-13-bruteforce-test.png

Secure ගැන කියන වචන

"Secure backend එක unknown username සහ wrong password දකෙටම එකම 401 status එක සහ එකම generic message එක භාවිතා කරනවා. User record එක නොතිබුණත් dummy hash එකක් check කරන නිසා timing difference එක අඩු කරනවා. Login route එක username සහ IP අනුව throttle කරලා repeated attempts 429 ලෙස refuse කරනවා. Regression tests දකෙම response equality සහ brute-force protection verify කරනවා. ඒ නිසා valid usernames leak වීමත් unlimited password spraying එකත් අවම කරනවා."

පනේවිය යුතු code point

```
$passwordValid = $user
    ? Hash::check($request->password, $user->password)
    : Hash::check($request->password, self::DUMMY_HASH);

if (! $user || ! $passwordValid) {
    return response()->json([
        'message' => 'The username or password is incorrect.',
    ], 401);
}
```

V-05 - Authentication Weaknesses

Original website

[Switch to the Original login page. Enter unknown . demo . user with a harmless incorrect password and submit.]

"I am now testing the login error behavior. First, I am using a username that does not exist."

[Show the response. Do not show the password.]

"The Original application returns a response that indicates that the username does not exist."

[Repeat with cashier . co lombo and the same incorrect password.]

"Now I am using a real username with an incorrect password. The response is different and indicates that the password is incorrect."

"These two different responses allow an attacker to distinguish valid usernames from invalid usernames. This is called username enumeration. The Original login route also did not have sufficient login-specific throttling, so repeated password attempts were not adequately limited."

[Show the Original authentication code.]

"The source code confirms that the unknown-user and wrong-password cases return different status codes or messages. This is the cause of the information leak."

Secure website

[Press Ctrl + Tab and open the Secure login page. Repeat the unknown-user attempt and the known-user wrong-password attempt.]

"I am now repeating the same two attempts against the Secure website. The password is never shown on screen."

"The Secure backend returns the same generic authentication response for both cases. An attacker can no longer learn whether a username exists from the login response."

[Show the secure authentication code.]

```
$passwordValid = $user
    ? Hash::check($request->password, $user->password)
    : Hash::check($request->password, self::DUMMY_HASH);

if (! $user || ! $passwordValid) {
    return response()->json([
        'message' => 'The username or password is incorrect.',
    ], 401);
}
```

"When the user does not exist, the backend still performs a dummy password-hash check. This helps reduce timing differences. Both invalid cases use the same generic message and HTTP 401 status."

[Show the login throttle route and the authentication tests.]

"The login route is also protected by a username-and-IP rate limit. Repeated attempts are refused with HTTP 429. The automated tests verify both generic error equality and brute-force protection."

Key code segments and file locations

Generic authentication response

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/AuthController.php Lines: 27 and 92-105

```
$passwordValid = $user
```

```

        ? Hash::check($request->password, $user->password)
        : Hash::check($request->password, self::DUMMY_HASH);

    if (! $user || ! $passwordValid) {
        return response()->json([
            'message' => 'The username or password is incorrect.',
        ], 401);
    }
}

```

Login throttling

File: demo/secure/pubudu-pos-api-secure/routes/api.php Lines: 61-66

```

Route::middleware('throttle:login')->group(function () {
    Route::post('/login', [AuthController::class, 'login']);
});

// Repeated attempts are refused with HTTP 429.

```

V-06 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කොටස කරන විදිහ

V-06 - Token and Session Lifecycle

මගේ live account එක deactivate නොකර code/test evidence වලින් පෙන්වන්න.

1. කලින් ගත් successful dashboard/session screenshot එක පෙන්වන්න:

V06-00-login-session-context.png

2. Original AuthController සහ config/sanctum.php පෙන්වන්න. Token create කරන code එක සහ 'expiration' => null පෙන්වන්න:

V06-01-original-session-code.png

3. Secure EnsureUserIsActive.php middleware එකමේ inactive user check, token delete සහ 403 response පෙන්වන්න:

V06-02-secure-active-middleware.png

4. Secure AuthController.php හි deactivation, role, password සහ username change වලදී \$user->tokens()->delete() කරන block එක පෙන්වන්න:

V06-03-secure-token-revocation.png

5. AuthenticationTest.php හි deactivation කල පසු token count 0 වන test එක පෙන්වන්න:

V06-04-secure-token-test.png

6. config/sanctum.php හි finite expiration value එක 480 minutes බව පෙන්වන්න:

V06-05-token-expiration.png

V-06 කියන වචන

"Original version එකමේ user login වුණාම bearer token එකක් ලැබුණා. User account එක පසුව deactivate කළත් කලින් නිකුත් කළ token එක revoke නොවූ අතර Sanctum expiration එකත් null වුණා. ඒ නිසා token එක දිගටම භාවිතා කළ හැකි අවදානම තිබුණා. Secure version එක protected request එකකට පෙර user active ද කියලා check කරනවා. Deactivate, role change, password change හෝ username change කළාම existing tokens delete කරනවා. තවද token එකට පැය අටක finite expiration එකක් දාලා තිබෙනවා. Test එක deactivation කල පසු කලින් තිබූ token count එක zero බව verify කරනවා."

V-06 - Token and Session Lifecycle

[Show the successful dashboard/session context, then the Original authentication code.]

"The final vulnerability in my assigned scope concerns token and session lifecycle management. In the Original version, a successful login creates a bearer token. The token expiration setting was unlimited, and an already-issued token was not automatically revoked when the user's account status or security-sensitive details changed."

[Show the Original token creation and the unlimited expiration setting.]

"This means that a token could remain usable for too long, even after the account should no longer be trusted."

[Switch to the Secure source code and show `EnsureUserIsActive.php`.]

"The Secure version checks that the user is active before allowing protected requests. If the account is inactive, the middleware removes the user's tokens and returns a forbidden response."

[Show the token-revocation block in the Secure authentication controller.]

"Tokens are also revoked when the account is deactivated or when security-sensitive values such as the role, password or username are changed."

[Show the authentication test and the finite Sanctum expiration setting.]

"The automated test confirms that the token count becomes zero after deactivation. The Secure configuration also uses a finite eight-hour token expiration instead of unlimited tokens. I am showing the code and test evidence rather than deactivating a seeded employee during the live demo."

Key code segments and file locations

Reject and revoke inactive-user tokens

File: `demo/secure/pubudu-pos-api-secure/app/Http/Middleware/EnsureUserIsActive.php` Lines: 45-69

```
if ($user && ! $user->is_active) {  
    $user->tokens()->delete();  
    return response()->json([  
        'message' => 'Account is inactive.',  
    ], 403);  
}
```

Finite token expiry

File: `demo/secure/pubudu-pos-api-secure/config/sanctum.php` Lines: 62-66

```
'expiration' => (int) env('SANCTUM_TOKEN_EXPIRATION', 480),  
  
// 480 minutes = 8 hours
```

OIDC - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාවස කරන විදිහ

Google OpenID Connect - Secure login evidence

1. Secure login page එකේ Sign in with Google button එක පෙන්වන්න. Screenshot:

OIDC-01-secure-google-button.png

2. Local Google client configuration නැතහොත් button එක click කරලා personal Google account එකකට යන්න එපා.
3. GoogleOidcService.php හි state, nonce, code_verifier, SHA-256 code_challenge සහ server-side cache storage පෙන්වන්න.
4. GoogleAuthController.php හි state consume කිරීම, code exchange කිරීම සහ ID token verify කිරීම පෙන්වන්න. Code screenshot එකම personal client secret, token හෝ Google account එක පෙන්වන්න එපා.

OIDC කියන වචන

"Secure Google sign-in එක Authorization Code flow සහ PKCE භාවිතා කරනවා. state value එක CSRF protection සඳහාත්, nonce replay protection සඳහාත්, code_verifier PKCE proof එක සඳහාත් භාවිතා කරනවා. Verifier සහ nonce browser එකට තැබීම server-side cache එකම තබනවා. Callback එකම state එක single-use ලෙස consume කරලා authorization code එක server-side exchange කරනවා. අවසානයේ ID token එකම signature, issuer, audience, expiry සහ nonce validate කරලා පමණක් staff account එකට sign in කරනවා."

English speech - read this in the video

Google OpenID Connect - Secure login evidence

[Show the Secure login page and the Sign in with Google button.]

"The Secure login page also includes Google OpenID Connect support. I will not use a personal Google account during this recording."

[Show the OIDC service and controller code. Keep secrets, tokens and personal account data hidden.]

"The authorization-code flow uses state for CSRF protection, nonce for replay protection, and a code verifier with a SHA-256 code challenge for PKCE. The verifier and nonce are stored on the server side. During the callback, the state is consumed only once, the authorization code is exchanged by the server, and the ID token is validated for its signature, issuer, audience, expiry and nonce before a staff account is authenticated."

"This prevents the application from trusting an unvalidated identity or a callback that was not created by the application."

Key code segments and file locations

Authorization Code + PKCE values

File: demo/secure/pubudu-pos-api-secure/app/Services/GoogleOidcService.php Lines: 75-107

```
$state = Str::random(40);
$nonce = Str::random(40);
$codeVerifier = Str::random(96);
$codeChallenge = rtrim(strtr(
    base64_encode(hash('sha256', $codeVerifier, true)), '+/', '-_'
), '=');

Cache::put($this->cacheKey($state), [
    'nonce' => $nonce,
    'code_verifier' => $codeVerifier,
], self::FLOW_TTL_SECONDS);
```

Callback validation

File: demo/secure/pubudu-pos-api-secure/app/Http/Controllers/Api/GoogleAuthController.php Lines: 102-131

```
$flow = $this->oidc->consumeFlow($request->string('state'));
// The server exchanges the code using the stored verifier.
// The ID token is checked for issuer, audience, expiry and nonce.
```

Final conclusion and recording checklist

Sinhala conclusion and checklist

අවසාන conclusion එක

“මම comparison එකෙන් පෙනෙන්නේ authentication එක පමණක් ඒරමාණවත් නොවන බවයි. Role authorization, department isolation, safe error messages, rate limiting සහ token lifecycle controls ද backend එකේ enforce කළ යුතුයි. Original version එකේ live browser result එකෙන් bug එක පෙන්වා, Secure version එකේ browser result, source code සහ automated regression test හුනෙන් fix එක verify කළා. V-04 Malithගේ assigned scope එකක් නිසා මම video එකේ මම V-04 claim කරන්නේ නැහැ.”

English conclusion

Conclusion

“This demonstration shows that authentication alone is not enough to secure an application. The system also needs role authorization, branch-level object authorization, safe authentication errors, rate limiting, token revocation and finite session lifetimes.

For each selected vulnerability, I showed the Original browser behavior, explained the security impact, demonstrated the Secure behavior, and connected the fix to source code and automated tests. The Original version exposes the weakness, while the Secure version enforces the control at the backend and verifies it with regression tests.

V-04 is part of Malith's assigned scope, so I have not claimed that vulnerability in this section. Thank you.”

Important: keep passwords masked. Do not display bearer tokens, client secrets or personal Google-account data. Do not submit the V-01 registration exploit and do not deactivate the seeded employee for V-06.